



GRAND LUXURY HOTEL & RESORT



Hotel Management System

Project Report

Project Title	Grand Luxury Hotel & Resort — Management System
Technology Stack	Java (Swing GUI), HTML5 & CSS3
Platform	Cross-Platform Desktop Application (JRE 8+)
Application Type	Single-File Desktop Application with Web Export
Source File	HotelManagement.java (828 lines, ~39 KB)
Date	July 2026

Placement Project — Short Project Submission • July 2026

Table of Contents

1. Abstract	3
2. Introduction	3
3. Problem Statement & Objectives	4
4. System Requirements	4
5. System Architecture & Design	5
6. Module Description	6
7. Implementation Details	7
8. User Interface Design	8
9. Code Walkthrough — Key Components	8
10. Data Flow Diagrams	9
11. Testing & Validation	10
12. Advantages & Limitations	11
13. Future Enhancements	11
14. Conclusion	12
15. References	12

1 Abstract

The **Grand Luxury Hotel Management System** is a single-file, self-contained Java desktop application that provides a comprehensive solution for hotel room booking, guest check-in/check-out management, billing and invoicing, and real-time dashboard analytics. Built using Java Swing with a premium custom-painted GUI, the application demonstrates object-oriented programming principles, event-driven architecture, and modern UI design patterns.

The project also includes an HTML/CSS web portal component that can be auto-generated from within the application, serving as a presentation layer for academic submissions. The system supports both in-memory data persistence and an optional MongoDB connection hook, making it adaptable for both demonstration and production-like environments.



Key Highlight: The entire application — GUI, business logic, data layer, and HTML export — is contained in a single Java file (828 lines), requiring zero external dependencies beyond the standard JDK.

2 Introduction

2.1 Background

The hospitality industry relies heavily on efficient management systems to handle guest bookings, room allocation, billing, and record-keeping. Traditional manual methods are error-prone, time-consuming, and difficult to scale. A computerized hotel management system eliminates these challenges by automating routine tasks, improving accuracy, and providing real-time visibility into hotel operations.

2.2 Purpose of the Project

This project aims to develop a **desktop-based Hotel Management System** using core Java technologies. It serves as both a functional prototype of a real-world hotel management solution and an academic project demonstrating:



Java Swing GUI

Custom-painted components with gradients, rounded buttons, card layouts, and hover animations



Object-Oriented Design

Encapsulation, inner classes, data modeling, and separation of concerns



Event-Driven Programming

ActionListeners, MouseListeners, and dynamic user interaction handling



HTML/CSS Integration

Automatic responsive web page generation from within the Java application

2.3 Scope

The application covers the core operational functions of a hotel front desk:

Function	Description
Room Booking	Register new guests with room type selection and price calculation
Check-In / Check-Out	Update guest status and generate tax invoices
Records Management	View all bookings in a sortable, styled table
Dashboard Analytics	Real-time statistics on bookings, occupancy, and revenue
HTML Export	Generate a web portal page for project presentation

3 Problem Statement & Objectives

3.1 Problem Statement

Hotels require an integrated system that handles room reservations, guest tracking, billing with tax computation, and operational analytics in a single, easy-to-use interface. Manual record-keeping leads to overbookings, billing errors, and lost revenue. There is a need for a lightweight, portable solution that does not require complex server infrastructure but can optionally connect to databases for persistent storage.

3.2 Objectives

- ▶ **Design and develop** a user-friendly Java Swing GUI application for hotel management with a premium, modern look and feel.
- ▶ **Implement** core hotel operations: room booking, check-in, check-out, and invoice generation with GST tax computation.
- ▶ **Create** an interactive dashboard displaying real-time statistics (total bookings, current occupancy, total revenue).
- ▶ **Build** custom UI components (gradient panels, rounded buttons, card panels, styled text fields) for a professional appearance.
- ▶ **Integrate** automatic HTML/CSS web page generation for project presentation and academic submission.
- ▶ **Provide** dual-mode data storage — in-memory `ArrayList` for immediate use and MongoDB driver detection for database connectivity.
- ▶ **Demonstrate** clean OOP design with inner classes, encapsulated data models, and separation of concerns.

4 System Requirements

4.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD equivalent or higher
RAM	2 GB (4 GB recommended)
Storage	50 MB free disk space
Display	1024 × 768 resolution or higher
Input Devices	Keyboard and Mouse

4.2 Software Requirements

Software	Version / Details
Operating System	Windows 10/11, macOS, or Linux
Java Development Kit (JDK)	JDK 8 or higher
Java Runtime Environment	JRE 8 or higher
Web Browser	Any modern browser (for HTML portal)
IDE (Optional)	IntelliJ IDEA, Eclipse, VS Code, or NetBeans
Database (Optional)	MongoDB 4.0+ with Java Driver

4.3 Software Dependencies

The project is designed as a **zero-dependency** application using only the Java Standard Library:

Package	Purpose
<code>javax.swing.*</code>	GUI framework — JFrame, JPanel, JTable, JButton, etc.
<code>java.awt.*</code>	Graphics, layout managers, event handling, 2D painting
<code>java.io.*</code>	File I/O for HTML generation
<code>java.time.*</code>	Date formatting with LocalDate and DateTimeFormatter
<code>java.util.*</code>	Collections (ArrayList, List)



No external JARs or build tools (Maven/Gradle) are required. The project compiles with a single `javac` command.

5 System Architecture & Design

5.1 Architectural Overview

The application follows a **single-file, monolithic architecture** with clear internal separation of concerns across four logical layers:



5.2 Design Principles Applied

Principle	Application in Project
Encapsulation	<code>Booking</code> inner class with private fields and public getters/setters
Single Responsibility	Each inner class handles one concern (e.g., <code>GradientPanel</code> only paints gradients)
Event-Driven Design	All user interactions handled through <code>ActionListeners</code> and <code>MouseListeners</code>
Custom Rendering	<code>paintComponent()</code> overrides for all custom UI components
Separation of Concerns	UI creation methods separated from business logic methods

5.3 Class Hierarchy

```
HotelManagement (extends JFrame)
├── Booking          - static inner class - Data Model
├── GradientPanel    - static inner class - extends JPanel
├── RoundedButton    - static inner class - extends JButton
│   └── Anonymous    - MouseAdapter (hover/press states)
├── CardPanel        - static inner class - extends JPanel
├── StyledTextField  - static inner class - extends JTextField
├── Anonymous Classes - ActionListeners, TableCellRenderers
└── Static Helpers   - createBadge(), createStatCard()
```


6 Module Description

The application is organized into **four main functional modules**, each represented as a tab in the user interface:

6.1 Module 1 — Room Booking

Purpose: Allows the front desk operator to create new guest reservations.



Guest Form

Full Name, Phone Number, Room Category dropdown, Stay Duration input



Price Calculation

Real-time total estimate based on room type × number of days



Auto-Generation

Unique Booking ID (from 1001), Room Number, Date stamp



Input Validation

Empty check, numeric validation, positive integer enforcement

Room Category	Rate / Night (₹)	Amenities Included
Standard	₹1,500	Wi-Fi, TV, AC, Work Desk
Deluxe	₹2,800	Balcony View, King Bed, Complimentary Breakfast
Executive Suite	₹5,000	Jacuzzi, Living Area, 24/7 Personal Butler

6.2 Module 2 — Check-In / Check-Out & Billing

Purpose: Manages guest status transitions and generates detailed invoices.

- ▶ Booking search by ID with instant invoice preview
- ▶ Status update to Checked-In or Checked-Out
- ▶ Detailed ASCII-art invoice with room charges, GST (12%), Service Charge (5%), and Grand Total
- ▶ Invoice rendered in monospaced font (Consolas) for precise alignment

Billing Formula

Subtotal = Room Rate × Number of Days
GST (12%) = Subtotal × 0.12
Service (5%) = Subtotal × 0.05
Grand Total = Subtotal + GST + Service Charge

6.3 Module 3 — All Booking Records

Purpose: Displays a comprehensive table of all bookings with 9 columns.

- ▶ Columns: ID, Guest Name, Phone, Room Type, Room #, Days, Total (₹), Date, Status
- ▶ Custom `DefaultTableCellRenderer` for alternating row colors
- ▶ Color-coded status: **Booked** **Checked-In** **Checked-Out**
- ▶ Non-editable cells, styled header with primary color background

6.4 Module 4 — Dashboard

Purpose: Provides at-a-glance operational analytics and system configuration.

- ▶ **Total Bookings** — Count of all reservations
- ▶ **Currently Checked-In** — Active guests count
- ▶ **Total Revenue** — Sum of all booking values (₹)
- ▶ System info: Java version, OS name, MongoDB connection status
- ▶ HTML export button to generate and launch `hotel.html`

7 Implementation Details

7.1 Project File Structure

```
Short_Project/ |— HotelManagement.java — Main source (828 lines, ~39 KB) |—
HotelManagement.class — Compiled main class |— HotelManagement$Booking.class — Data Model
inner class |— HotelManagement$GradientPanel.class |— HotelManagement$RoundedButton.class
|— HotelManagement$RoundedButton$1.class — MouseAdapter |— HotelManagement$CardPanel.class
|— HotelManagement$StyledTextField.class |— HotelManagement$1.class ... $4.class — Anonymous
listeners |— hotel.html — Web portal (43 lines) |— README.md — Documentation
```

7.2 Compilation & Execution

```
# Step 1: Compile the source file
javac HotelManagement.java

# Step 2: Launch the desktop GUI application
java HotelManagement

# Step 3: Open the web portal (auto-generated or manual)
# Open hotel.html in any web browser
```

7.3 Design System — Color Palette

PRIMARY #1E3C72	PRIMARY_LIGHT #2A5298	ACCENT #009688	SUCCESS #2E7D32	DANGER #C62828
WARNING #F57C00	GOLD #FFC107	ACCENT_DARK #00796B	PANEL_BG #F0F2F7	TEXT #212121

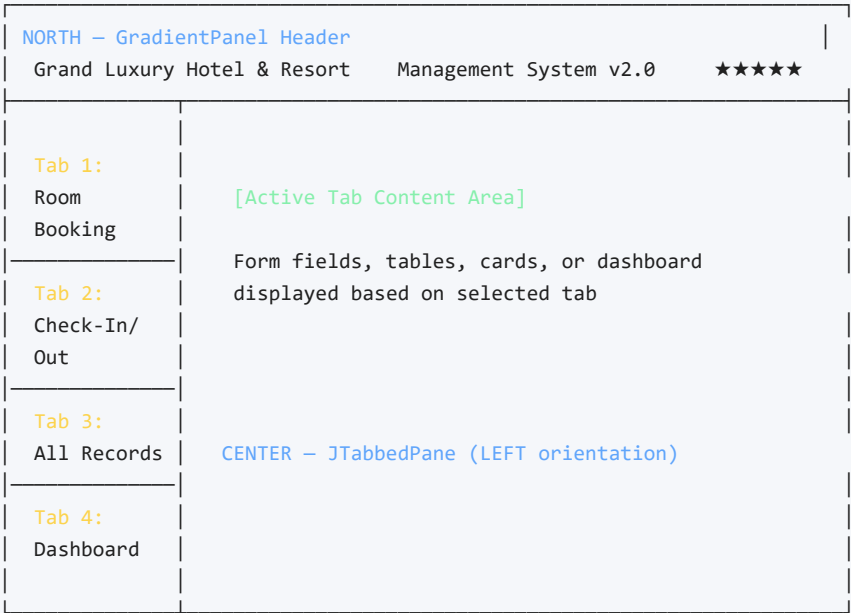
7.4 Typography System

Constant	Font Family	Style	Size	Usage
<code>FONT_TITLE</code>	Segoe UI	Bold	22pt	Section titles
<code>FONT_SUBTITLE</code>	Segoe UI	Plain	14pt	Subtitles, descriptions
<code>FONT_LABEL</code>	Segoe UI	Bold	13pt	Form labels
<code>FONT_INPUT</code>	Segoe UI	Plain	13pt	Text fields, dropdowns
<code>FONT_BUTTON</code>	Segoe UI	Bold	13pt	Button text
<code>FONT_TABLE</code>	Segoe UI	Plain	12pt	Table content
<code>FONT_MONO</code>	Consolas	Plain	13pt	Invoice display

8 User Interface Design

8.1 Main Window Layout

The application window is sized at **960 × 680 pixels** (minimum 800 × 550) and uses a `BorderLayout` :



8.2 Custom UI Components



GradientPanel

JPanel with linear `GradientPaint` background. Uses `Graphics2D` with antialiasing. Paints diagonal gradient from PRIMARY to PRIMARY_LIGHT.



RoundedButton

JButton with 12px rounded corners via `RoundRectangle2D`. Three states: default, hover (brighter), pressed (darker). Shadow effect on hover.



CardPanel

JPanel with soft drop shadow (3px offset), white rounded rectangle background (16px radius), and consistent 18×20px internal padding.



StyledTextField

JTextField with light background (#F8F9FC), rounded line border in soft blue-gray, and 6×10px internal padding for comfortable input.

9

Code Walkthrough — Key Components

9.1 Data Model — Booking Class

```
public static class Booking {  
    private static int idCounter = 1001;    // Auto-increment ID  
    private int bookingId;  
    private String guestName, phone, roomType, status;  
    private int roomNumber, days;  
    private double totalPrice;  
    private String bookingDate;  
  
    // Constructor auto-assigns ID, sets status to "Booked",  
    // and records current date via LocalDate.now()  
}
```



Design Decision: The `idCounter` is a `static` field shared across all instances, ensuring unique sequential booking IDs starting from 1001. This eliminates the need for a database-generated sequence.

9.2 Invoice Generation

The `generateBillText()` method produces a formatted ASCII-art invoice using Unicode box-drawing characters:

```
┌────────────────────────────────────────────────────────────────────────────────┐  
│                                GRAND LUXURY HOTEL & RESORT                       │  
│                                OFFICIAL INVOICE                               │  
├────────────────────────────────────────────────────────────────────────────────┤  
│ Booking ID      : 1001                                                       │  
│ Guest Name     : Rahul Sharma                                                │  
│ Phone          : 9876543210                                                  │  
│ Room Category  : Deluxe                                                       │  
│ Room Number    : #101                                                         │  
│ Stay Duration  : 2 Days                                                       │  
├────────────────────────────────────────────────────────────────────────────────┤  
│ Room Charges   : ₹ 5,600.00                                                  │  
│ GST (12%)      : ₹ 672.00                                                    │  
│ Service Charge : ₹ 280.00                                                    │  
├────────────────────────────────────────────────────────────────────────────────┤  
│ GRAND TOTAL    : ₹ 6,552.00                                                  │  
└────────────────────────────────────────────────────────────────────────────────┘
```

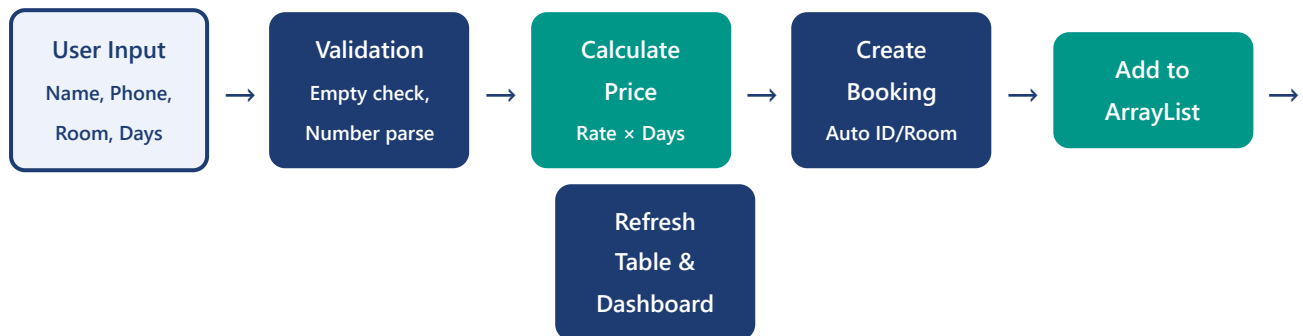
9.3 MongoDB Integration Hook

```
private void checkMongoConnection() {  
    try {  
        Class.forName("com.mongodb.client.MongoClients");  
        mongoAvailable = true;    // Display: CONNECTED  
    } catch (ClassNotFoundException e) {  
        mongoAvailable = false;    // Display: IN-MEMORY MODE  
    }  
}
```

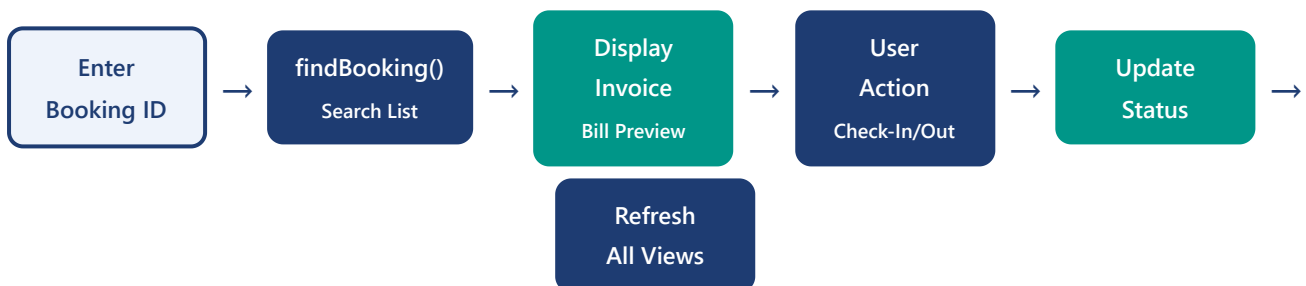
The system uses **reflection-based driver detection** (`Class.forName`) to check if the MongoDB Java driver is available on the classpath without creating a hard compile-time dependency.

10 Data Flow Diagrams

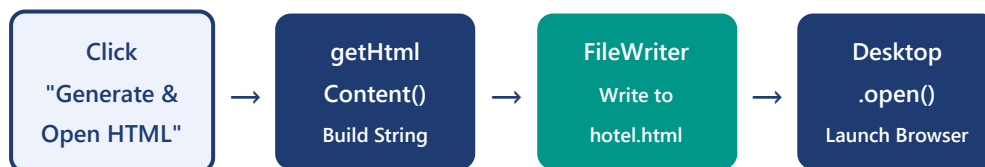
10.1 Booking Creation Flow



10.2 Check-In / Check-Out Flow



10.3 HTML Export Flow



10.4 Class Relationship Summary

Class	Extends	Role	Key Methods
HotelManagement	JFrame	Main application frame	main(), handleBooking(), calculatePrice()
Booking	—	Data model (POJO)	Getters, setters, auto-ID
GradientPanel	JPanel	Gradient-painted header	paintComponent()
RoundedButton	JButton	Styled button with hover	paintComponent(), MouseAdapter
CardPanel	JPanel	Card with shadow	paintComponent()
StyledTextField	JTextField	Rounded input field	Constructor styling

11 Testing & Validation

11.1 Test Cases

#	Test Case	Input	Expected Output	Result
1	Valid booking creation	Name: "John", Phone: "9876543210", Standard, 3 days	Booking ID 1003, Total ₹4,500	Pass
2	Empty name validation	Name: "", Phone: "123"	Warning: "Provide Guest Name and Phone"	Pass
3	Invalid days (text)	Days: "abc"	Error: "Enter a valid number of days"	Pass
4	Zero days input	Days: "0"	Error: "Enter a valid number of days"	Pass
5	Negative days	Days: "-2"	Error: "Enter a valid number of days"	Pass
6	Price — Standard	Standard, 5 days	₹7,500.00	Pass
7	Price — Deluxe	Deluxe, 2 days	₹5,600.00	Pass
8	Price — Executive	Executive Suite, 1 day	₹5,000.00	Pass
9	Search — valid ID	ID: 1001	Invoice for "Rahul Sharma"	Pass
10	Search — invalid ID	ID: 9999	Error: "Booking ID not found!"	Pass
11	Check-In update	ID: 1001 → Mark Checked-In	Status = "Checked-In", table refreshes	Pass
12	Check-Out & invoice	ID: 1001 → Check-Out	Status = "Checked-Out", invoice generated	Pass

#	Test Case	Input	Expected Output	Result
13	Dashboard stats	After 2 sample bookings	Total: 2, Revenue: ₹20,600	Pass
14	HTML generation	Click "Generate & Open"	hotel.html created, opens in browser	Pass
15	No MongoDB driver	No driver on classpath	Status: "IN-MEMORY MODE"	Pass

11.2 Invoice Tax Calculation Verification

Sample: Booking ID 1001 — Deluxe Room, 2 Nights

Component	Calculation	Amount (₹)
Room Charges (Subtotal)	₹2,800 × 2	5,600.00
GST (12%)	₹5,600 × 0.12	672.00
Service Charge (5%)	₹5,600 × 0.05	280.00
Grand Total		6,552.00

12 Advantages & Limitations

12.1 Advantages

- ✓ **Zero Dependencies** — Runs with just the standard JDK; no external libraries, build tools, or servers required.
- ✓ **Single-File Architecture** — The entire application (828 lines) is in one `.java` file, easy to distribute and submit.
- ✓ **Premium UI Design** — Custom-painted components with gradients, rounded corners, shadows, and hover animations.
- ✓ **Dual Storage Mode** — Seamlessly falls back to in-memory when MongoDB is unavailable.
- ✓ **Cross-Platform** — Runs on Windows, macOS, and Linux with any JRE 8+ installation.

- ✓ **Auto HTML Export** — Generates a responsive web portal from within the application.
- ✓ **Real-Time Dashboard** — Provides instant analytics without manual report generation.
- ✓ **Professional Invoices** — Formatted ASCII-art invoices with GST + service charge calculations.

12.2 Limitations

#	Limitation	Impact
1	No persistent storage by default	In-memory data lost on application close
2	Single-user operation	No multi-user or network access support
3	No authentication system	No login or role-based access control
4	Fixed room pricing	Rates are hardcoded; no admin panel for dynamic pricing
5	No date-based availability	Room assignments are sequential; no conflict detection
6	Deprecated applet container	HTML applet tag non-functional in modern browsers

13 Future Enhancements

Enhancement	Description	Complexity
Database Integration	Full MongoDB / MySQL CRUD for persistent data storage	Medium
User Authentication	Login screen with admin/receptionist roles and password hashing	Medium
Room Calendar	Date-range booking with conflict detection and visual calendar	High
Email Notifications	Send booking confirmation and invoices via SMTP	Medium
PDF Invoice Export	Generate downloadable PDF invoices using iText or PDFBox	Medium
Search & Filter	Filter bookings by date range, status, room type, or guest name	Low
i18n Support	Multi-language internationalization support	Medium
REST API Layer	Expose operations via Spring Boot REST endpoints	High
Mobile Portal	Enhance HTML portal with Bootstrap for mobile responsiveness	Medium
Report Charts	Occupancy reports, revenue charts, and trend analysis	Medium

14 Conclusion

The **Grand Luxury Hotel Management System** successfully demonstrates the development of a fully functional, visually polished desktop application using core Java technologies. The project achieves all stated objectives:

- ✓ A **premium Swing GUI** with custom-painted components, gradient effects, and card-based layouts

- ✓ Complete **room booking workflow** with input validation and automatic ID generation
- ✓ **Check-in/check-out management** with detailed GST-inclusive invoice generation
- ✓ An interactive **records table** with color-coded statuses and alternating row styling
- ✓ A real-time **analytics dashboard** with key metrics (bookings, occupancy, revenue)
- ✓ Automatic **HTML/CSS web portal generation** for academic presentation
- ✓ **Dual-mode storage** with graceful fallback from MongoDB to in-memory mode

The single-file architecture (828 lines of Java) makes the project exceptionally portable and easy to evaluate, while the quality of the custom UI components demonstrates proficiency in Java 2D graphics, event-driven programming, and modern design principles. The project serves as a strong foundation that can be extended with database persistence, authentication, and REST APIs to become a production-ready hotel management solution.

15

References

#	Reference
1	Oracle Corporation. (2024). <i>Java Swing Tutorial</i> . https://docs.oracle.com/javase/tutorial/uiswing/
2	Oracle Corporation. (2024). <i>Java 2D Graphics API</i> . https://docs.oracle.com/javase/tutorial/2d/
3	Oracle Corporation. (2024). <i>java.time Package Documentation</i> . https://docs.oracle.com/javase/8/docs/api/java/time/
4	MongoDB Inc. (2024). <i>MongoDB Java Driver Documentation</i> . https://www.mongodb.com/docs/drivers/java/
5	Horstmann, C. S. (2019). <i>Core Java Volume I — Fundamentals</i> (11th Ed.). Pearson Education.
6	Deitel, P., & Deitel, H. (2020). <i>Java: How to Program, Early Objects</i> (11th Ed.). Pearson.
7	Mozilla Developer Network. (2024). <i>HTML5 & CSS3 Documentation</i> . https://developer.mozilla.org/
8	Government of India. (2017). <i>Goods and Services Tax (GST) Act</i> . CBIC.

— End of Report —

Grand Luxury Hotel Management System • Project Report • July 2026